API integration design

Oct 10, 2026 · @Abubakar

Summary

The stack.

- **Society accounts.** Each society's money sits in a society-named wallet at Embedly (Sterling), with ProvidusUnity or Wema as the second bank. Larger payouts go through the society's own corporate account at GTBank or FirstBank, where its signatories approve.
- **Collections.** Member transfers come in through Monnify reserved accounts that split straight to the society's account. Card and direct-debit payments use Paystack sub-accounts.
- **Payroll check-off.** Employers pay into employer-specific collection accounts. Remita is used for government payrolls.
- **The platform's own float.** It sits in a Paystack balance.

How it fits together.

- Every bank sits behind one adapter layer with a single contract.
- Every callback enters through one verified webhook gateway.
- Money only moves on an approved payout with the platform's own reference.
- The ledger posts only when a callback and a status query agree.
- The nightly statement proves each society's books against its bank.

The process. Integration runs in five phases:

1. Commercial and legal.
2. KYB of the platform at each provider.
3. Sandbox build.
4. Certification.
5. Production pilot.

Embedly, Paystack and Monnify sandboxes are self-serve. ProvidusUnity, Wema and Remita are sales-led and need certification with the provider.

What is not public. No provider publishes a certification test pack, an uptime commitment or a notice period for API changes. No bank publishes its bulk-payment file template. These must be obtained in the commercial phase.

The right providers and banks, by capability

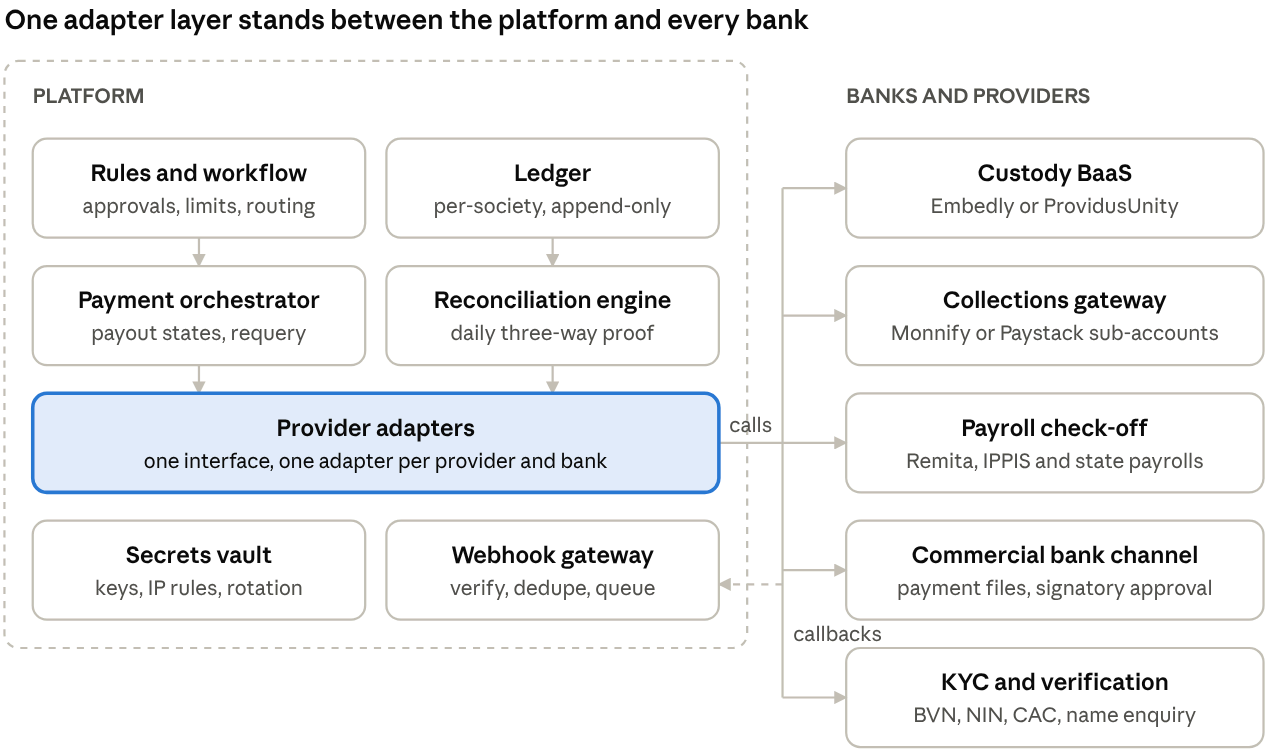
Capability	Primary	Fallback	Why
Society accounts, automated route	Embedly (Sterling)	ProvidusUnity or Wema	A commercial bank with ₦5m deposit cover. Corporate onboarding by API, including the board resolution. Each wallet gets its own account number and a history API. Wema's approval callback adds a second check
Society accounts, signatory route	The society's corporate account at GTBank or FirstBank	Access, Zenith or UBA	Maker-checker with tokens and per-user limits already exist. No API: the platform prepares payment files
Member contributions by transfer	Monnify reserved accounts, one per member, split to the society's account	ProvidusUnity reserved accounts, if the society's account is at Providus	Each member's payment is identified by its account number. The split sends it straight to the society's account, the same day
Card and direct debit	Paystack, with one sub-account per society and a zero platform share	Monnify	Next-day settlement to the society's own account

Capability	Primary	Fallback	Why
Payroll check-off remittances	One collection account per employer at the society's bank, plus Remita for government payrolls	Remita alone	The employer's money lands in the society's account and is matched to the deduction schedule
Beneficiary name check	The custody bank's name enquiry	Paystack account resolve	It runs before every payout
Platform operating float	Paystack balance	Monnify wallet	Paystack documents low-balance webhooks

Not for society money.

- **VFD and Anchor.** Both are microfinance banks or sit at one, so deposit cover is only ₦2m.
- **Mono Disburse.** It needs each society to be Mono's own client.
- **Open-banking payment initiation.** It is not live in Nigeria.

Integration architecture



integration architecture · six platform modules, five external connections

How the modules fit.

- The rules engine and the ledger decide what should happen. The orchestrator and the reconciliation engine make it happen and prove it.
- Nothing in the platform calls a bank directly. Every call goes through an adapter that speaks one internal language and translates it for one provider.
- Every callback enters through one webhook gateway. The gateway verifies each callback and queues it before anything else reads it.
- Adding a second bank means writing one adapter, not changing the workflow.

The adapter contract

Every adapter implements the same operations. A provider that lacks one returns "not supported", and the router picks another route.

Operation	What it does	Embedly	ProvidusUnity	Wema	Paystack or Monnify	Bank channel
Register society	Creates the society as a corporate customer	Corporate customer with RC number, TIN, directors' BVN and documents	No account-opening API. Opened at the bank	Account creation, details to confirm	Sub-account with the society's bank account	Opened at the branch
Open account	Returns the society's account number	Corporate wallet, which returns its own account number	Existing account	Wallet or account	Not applicable	Existing account
Collection account	One account number per member or employer	Wallet account number	Dynamic or reserved accounts that settle to the society's account	Wallet	Reserved accounts or dedicated virtual accounts, with a split	Not applicable
Name enquiry	Confirms the beneficiary's name	Provider check, details to confirm	Name enquiry service	Name enquiry service	Account resolve	Bank checks at upload
Send payout	Moves money from the society's account	Interbank transfer from the society's wallet	Transfer from a named debit account	Debit-wallet request, gated by the platform's approval callback	Not used for society money	Payment file for signatories
Payout status	Final answer for one payout	Status endpoint and payout webhook	Transaction status query	Status query	Not used	Execution report
Balance	Current balance	Wallet balance	Balance enquiry	Balance enquiry	Not used	Statement
Transactions	Statement lines for a date range	Paged wallet history	Statement endpoint	Statement service	Settlement reports	Statement file

Operation	What it does	Embedly	ProvidusUnity	Wema	Paystack or Monnify	Bank channel
Verify callback	Proves a webhook is genuine	Signature header, scheme to confirm	Credentials and settlement ID, then status query	Debit and credit notices, signing to confirm	Signature header with HMAC-SHA512	Not applicable

Rules for every adapter.

- Map provider codes into a small internal set: success, failed, pending, reversed. An unknown code maps to pending, never to failed.
- Pass the payout's own identifier as the provider reference.
- Log the raw request and response, with personal data masked.
- Never retry a payout on its own. Return the result to the orchestrator.

The integration flows

1. Society onboarding.

1. The society's officers upload their documents in the platform:
   - registration certificate and TIN;
   - board resolution;
   - bylaws or memorandum and articles;
   - SCUML certificate;
   - each signatory's ID and BVN.
2. The platform checks the signatories' BVNs, then registers the society as a corporate customer at the custody bank and uploads the documents.
3. The bank reviews the business documents and sends an approval webhook.
4. The platform opens the society's wallet. It checks that the returned account name matches the society's registered name, and refuses the link if it does not.
5. The platform creates the society's books, and maps the bank account to the ledger's bank account.
6. The platform creates the society's sub-accounts at the gateways, settling to the new account with a zero platform share.
7. Paystack holds the first payout to a new sub-account until it is verified in the dashboard, so an operator verifies it.

8. The three-party agreement, the resolution and the payout caps are stored. The society goes live.

2. Member contributions by transfer.

1. The platform reserves one account number per member at Monnify. The income split sends 100% to the society's sub-account.
2. The member transfers to that number. The gateway sends a signed webhook.
3. The platform verifies the webhook, confirms the transaction by query, and credits the member's savings as pending.
4. The settlement report confirms the money reached the society's account. The entry is then posted, and the member gets a receipt.

3. Card and direct-debit contributions.

1. The platform starts the payment with the society's Paystack sub-account, with the society bearing the fee.
2. For a recurring debit, the stored authorisation is charged on schedule.
3. A success webhook, confirmed by query, credits the member as pending. Next-day settlement posts it.

4. Payroll check-off remittance.

1. The platform produces each employer's deduction schedule in the employer's or Remita's format.
2. The employer pays into its own collection account at the society's bank. The credit webhook and statement line are captured.
3. The platform imports the remittance advice and matches it line by line to the schedule.
4. Matched lines credit members. Short or missing lines go to employer arrears and are aged.

5. Automated payout through the custody bank.

1. The workflow approves the payout and places the hold, as in the earlier tabs.
2. The adapter runs a name enquiry and checks the name against the member's KYC name.
3. The adapter sends an interbank transfer from the society's wallet, with the payout identifier as the customer reference.
4. The provider answers "being processed". The payout moves to sent.

5. A payout webhook arrives. The gateway verifies it, and the orchestrator confirms it by status query before posting the ledger.
6. Embedly stops webhooks after five retries, so a scheduled sweep queries every payout still pending.
7. Embedly documents no idempotency. Before any resend, the orchestrator must find the original by its reference and prove that it failed.

With Wema, the bank calls the platform's approval URL before debiting. The platform answers yes only if the payout carries its full set of approvals. This is a second check, but it is still the platform's own control.

6. Payout through the bank's signatory channel.

1. The platform builds the bank's bulk payment file from approved payouts, and records the file's hash and totals.
2. An authorised uploader loads it into the society's corporate banking profile. The signatories approve with their tokens.
3. The bank executes the payments. The platform imports the execution report and the statement.
4. Each executed line is matched to its payout. A line that was added, removed or changed is flagged before the ledger is posted.

7. Statement sync.

1. Every night the platform pulls each account's transactions for the day: wallet history, statement endpoints or uploaded statement files.
2. Lines are converted to one internal format and fed to the matching engine.
3. Balances are pulled hourly to detect drift early.

8. Webhook intake.

1. Every provider has its own endpoint. The signature is checked first. Where a provider does not sign, the source IP is checked and the event is always confirmed by query.
2. The raw event is stored, and duplicates are dropped by event ID or reference.
3. The platform acknowledges only after the event is safely stored. ProvidusUnity releases settlement only on acknowledgement, so the platform replies with its success code after storing, and with its duplicate code for repeats.
4. Processing happens from the queue. The event never posts the ledger on its own word.

Technical and security requirements

Area	Requirement
Credentials	Every key and password sits in a secrets vault, never in code or config files. Keys are rotated every quarter and at once after an incident. Separate keys are used for test and live
Network	Outbound calls leave from fixed IP addresses that providers can whitelist. Inbound webhooks are accepted only from the provider's published addresses, where it publishes them
Transport	TLS 1.2 or higher on every call. Certificate checks are never switched off. Mutual TLS or VPN where a bank requires it
Callback integrity	Every webhook's signature is verified with a constant-time comparison. Unsigned callbacks are confirmed by status query
Idempotency	Every outbound money call carries the platform's own unique reference, and every retry reuses it. Every inbound event is processed once, keyed by its ID
Timeouts and retries	Read-only calls retry with backoff. Money-moving calls are never auto-retried. A timeout moves the payout to pending
Rate limits	Each adapter respects the provider's limits with a queue. Bulk payouts are batched to the provider's maximum
Logging	Every request and response is logged with a correlation ID. Account numbers and BVNs are masked, and logs are kept for five years for AML
Data protection	BVN and account data stay in Nigeria. They are encrypted at rest, and only roles that need them can read them

Area	Requirement
Availability	Webhook intake stays up during deployments. A provider outage queues work instead of losing it
Change control	Adapter changes are tested against the provider's sandbox in CI before release. Provider API changes are tracked against the provider's notice periods

The integration process, from first call to go-live

Researched 10 October 2026. Several provider sites blocked automated reading, so some points rest on search extracts. Full notes are in integration_onboarding_golive.md.

Phase 1. Commercial and legal.

1. Hold sales calls with each provider, using the open questions below.
2. Sign an NDA, then the master service agreement, the service levels and a data processing agreement.
3. Get counsel's opinion on the automated route, and agree the three-party agreement template.
4. Get each bank's bulk-payment file format and sample schedule. These are given under contract, not published.

Phase 2. KYB of the platform at each provider.

- **All providers.** Expect to supply:
  - the CAC documents and the directors' IDs and BVNs;
  - an AML and CFT policy, and a KYC and customer due diligence policy;
  - the SCUML certificate;
  - evidence of data protection compliance;
  - a security questionnaire.
- **Embedly** requires KYC, AML and CFT checks and third-party due diligence before go-live. It shows production keys only after its administrator approves the documents. No CBN licence is stated as required.
- **Monnify** adds an operating licence and SCUML for fintech-sector merchants (secondary source).

- **Remita** needs owners holding 5% or more declared, and the top KYC level for every user.
- **Paystack** activates a business in one to three days (secondary source). Cooperative societies are a recognised entity type for transfers.

Phase 3. Sandbox build.

- **Self-serve.**
  - **Embedly.** The staging site approves dummy documents at once. Its current base addresses are waas-staging.embedly.ng and waas-prod.embedly.ng. An older doc set points elsewhere, so confirm which one the contract covers.
  - **Paystack and Monnify.** Test keys are issued on sign-up.
- **Sales-led.**
  - **ProvidusUnity.** Request keys through a form, and the bank sets up the platform on its side. Its staging site runs over plain HTTP on a bare IP address, so never send real data to it.
  - **Wema.** Subscribe to the product, then apply for production credentials. Every call carries a channel ID. Wema also registers the platform's approval callback address, even for testing.
- **Build all eight flows against sandboxes.** Run the test plan in continuous integration.

Phase 4. Certification.

- **No published test packs.** No provider publishes a test pack, so send the platform's own test evidence for sign-off.
- **ProvidusUnity rules.**
  - Re-query a transfer's status for 15 minutes.
  - Dynamic accounts expire after 10 minutes by default, and at most after 48 hours.
  - Reserved accounts come from a limited pool, so ask for enough for every member.
- **Remita.** Deduction at source and aggregator collections need certification with SystemSpecs. Production deduction at source needs a lending licence, so whether a cooperative qualifies is the first question.

Phase 5. Production set-up and pilot.

1. **Fixed IP addresses.** Register them with every provider. Monnify and Remita require this for payouts.
2. **Monnify.** Ask Monnify's integration support by email to switch off the payout OTP once server-side approvals are in place.

3. **Paystack.**
    - Turn on app-based two-factor login for every dashboard user.
    - Turn on transfer approvals and IP whitelisting, which also unlocks higher limits.
    - Rate limits are about 600 calls a minute in live mode, and 900 a minute for starting transfers (secondary source).
4. **Keys and webhooks.** Load keys into the vault and register the webhook addresses.
5. **Live tests.** Send small live test transfers to staff accounts. Prove every webhook, status query and statement line.
6. **Pilot.** Go live with one society on the signatory route. Turn on the automated route below a small cap after a clean month.

Setting up the bank's signatory channel, per society.

- **The request.** The society sends a signed letter on its letterhead asking the bank to add the platform's operator as uploader. Access activates profiles within 24 hours.
- **GTBank's costs.** About ₦2,500 per approver token and ₦15 per bulk item.
- **File formats.**
    - GTBank's channel lets the customer configure its own file format.
    - FirstBank hands over its format under contract.
    - Access offers an Excel template with a bank-code list inside its portal.
    - Zenith uses an eight-field file for salaries and vendors.
- **A neutral alternative.** NIBSS Pay takes CSV files of up to 100,000 items, with uploader, approver and auditor roles.
- **Host-to-host.** GTBank mentions a web-service API, FirstBank's terms describe a file dropped in a set location, and Zenith offers SWIFT messages. These suit larger volumes later.

Test plan

Each case runs in the provider's sandbox first, then again with small live amounts before go-live.

Area	Cases
Onboarding	Society created and approved. Approval rejected. Wallet name mismatch refused. Sub-account created and verified
Collections	Payment to a member's reserved account. Wrong amount. Payment to a closed account. Duplicate webhook. Webhook with a bad signature. Settlement report missing a line
Payroll	A full remittance matching the schedule. A short remittance. A remittance with no advice. Two employers paying into the wrong accounts
Payouts, automated	Success. Insufficient funds. Invalid account. Name mismatch. Timeout with no answer. Webhook lost, so the sweep finds it. Late reversal after success. Resend attempted on a pending payout and blocked
Payouts, bank channel	File accepted and approved. File rejected by the bank. A line edited in the bank channel and caught. Partial execution
Reconciliation	A bank debit with no order. An order with no bank line. A balance difference. An unmatched credit sent to suspense
Security	An expired key. A rotated key. A call from an address not on the whitelist. A replayed webhook. The kill switch stopping payouts within one minute
Resilience	A provider outage queues payouts. Failover to the second bank. Webhook intake surviving a deployment

Acceptance criteria

AC-INT-01 Society onboarding.

- *Given* a society with approved documents,

- *when* the custody bank opens its wallet,
- *then* the wallet's name matches the society's registered name, the ledger bank account is created, and the gateway sub-accounts settle to that wallet with a zero platform share.

AC-INT-02 Name mismatch.

- *Given* a wallet whose returned name differs from the registered name,
- *when* the platform tries to link it,
- *then* the link is refused and an operator is alerted.

AC-INT-03 Member transfer.

- *Given* a member's reserved account,
- *when* the member transfers money to it,
- *then* the member's savings show the credit as pending within one minute of the webhook, and as posted once settlement is confirmed.

AC-INT-04 Duplicate or forged webhook.

- *Given* a webhook already processed, or one with a bad signature,
- *when* it arrives,
- *then* it changes nothing in the ledger, and the forged one raises a security alert.

AC-INT-05 Payroll remittance.

- *Given* a deduction schedule,
- *when* the employer's remittance and advice arrive,
- *then* every matched line credits the right member, and every unmatched line appears in employer arrears with its age.

AC-INT-06 Automated payout success.

- *Given* an approved payout under the society's cap,
- *when* the bank confirms success by webhook and status query,
- *then* the ledger posts once, the hold is released into a debit, and the member gets a receipt with the session ID.

AC-INT-07 Timeout.

- *Given* a payout whose request timed out,
- *when* there is no final answer,

- *then* the payout stays pending, no second transfer is sent, and the sweep settles it by status query or escalates it after 24 hours.

AC-INT-08 Late reversal.

- *Given* a payout posted as successful,
- *when* the bank reports a reversal,
- *then* a reversing entry is posted, the payout reopens, and the society is notified.

AC-INT-09 Bank-channel payout.

- *Given* an approved batch above the cap,
- *when* the platform builds the file,
- *then* the file's totals equal the approved batch.
- *When* the bank's execution report is imported,
- *then* every line is matched, and any added, removed or changed line blocks posting until a person reviews it.

AC-INT-10 Daily proof.

- *Given* a day's activity,
- *when* the nightly sync runs,
- *then* the ledger's bank balance equals the bank's posted balance for every society, or a break is opened with an owner.

AC-INT-11 Unexplained debit.

- *Given* a bank debit with no payout behind it,
- *when* reconciliation finds it,
- *then* that society's outbound payouts are suspended, and the chair and the CTO are alerted within one hour.

AC-INT-12 Failover.

- *Given* the primary custody provider is down,
- *when* payouts are due,
- *then* they queue without loss. Societies that have a second bank set up route there, and nothing is sent twice.

AC-INT-13 Kill switch.

- *Given* an authorised person triggers the kill switch,
- *when* payouts are queued or in progress,

- *then* no new transfer leaves any provider within one minute, and the directors are alerted.

Operating the integration

What to watch.

Signal	Alert when
Webhook delay per provider	No events for 30 minutes in business hours, or a delay over 5 minutes
Pending payouts	Any older than 1 hour, or more than 20 at once
Provider error rate	Above 5% of calls in 15 minutes
Reconciliation breaks	Any critical break, or any high break older than one day
Balance drift	The ledger's bank balance differs from the bank's hourly balance
Name-check failures	A sudden rise for one society, which may signal fraud
Key expiry	30 days before any credential or certificate expires

Runbooks to have ready.

- Provider down: pause sends and queue, tell societies, switch route where possible.
- Webhooks stopped: run the status sweep and contact the provider.
- Unexplained debit: hit the kill switch for the society, call the bank's fraud desk, preserve evidence.
- Key leak: rotate the key, block the old one, review 30 days of calls.
- Bank file rejected: correct and regenerate with a new file hash, never edit by hand.

What to negotiate with each provider.

- Uptime of at least 99.9%, published status and incident notices.
- A named technical contact and a fraud-desk line answered around the clock.
- At least 90 days' notice of breaking API changes.
- Webhook retry policy and a replay endpoint.

- Daily statement access that survives the end of the contract.

Open questions and next steps

For providers.

1. **Embedly.**
   - Which API stack and webhook signing scheme does the contract cover?
   - Does a resend with the same reference get rejected?
   - What are the corporate wallet limits?
   - Do you accept a state cooperative certificate without a CAC number?
2. **ProvidusUnity.**
   - How many reserved accounts can each society have?
   - Can the settlement account be the society's own account?
   - Is there a debit webhook?
3. **Wema.**
   - What are the base addresses, and the corporate wallet terms?
   - Can the approval callback carry the society's approvals as evidence?
4. **Monnify.**
   - Confirm same-day settlement to a society's account at Sterling.
   - How many reserved accounts may each society have, and at what fees?
5. **Paystack.**
   - Confirm the zero platform share on a sub-account split.
   - Confirm the transfer limits for the float.
6. **Remita.** Can a cooperative use deduction at source without a lending licence, or through a licensed partner?
7. **GTBank and FirstBank.**
   - Can one platform company be the uploader on many unrelated societies' profiles?
   - What are the bulk file format and the row limit?
8. **All providers.** What are the uptime commitments, the change-notice periods, and the fraud-desk contacts?

Next steps.

1. Write the adapter contract as code, and build the Embedly and Paystack adapters first against their self-serve sandboxes.

2. Build the webhook gateway, the orchestrator states and the nightly sync before any live money moves.
3. Book sales calls with ProvidusUnity, Wema, Monnify and Remita with the questions above.
4. Get one bank's bulk file format from a pilot society's relationship manager, and build the file generator.
5. Run the test plan, collect evidence, and complete certification.
6. Pilot with one society on the signatory route.

Full notes are in the research folder, in integration_onboarding_golive.md and the earlier notes it builds on.